

Artificial Intelligence for Digital Characters

Exercise 3: Speech Synthesis

Hand-out Date: 22 April 2026



Goals

- Understand the key concepts in speech synthesis, including text normalization, prosody, prominence, and different synthesis techniques.
- Understand the architecture and functionality of acoustic models in text-to-speech (TTS) systems, including Tacotron 2 and FastSpeech.
- Explore vocoder techniques for waveform generation, comparing classical signal processing methods and modern neural vocoders such as WaveNet, HiFiGAN, and DiffWave.
- Analyze methods for controlling speaker identity and speaking style in TTS using one-hot labels, learned embeddings, and style reference encoders.
- Develop an understanding of zero-shot and few-shot adaptation techniques for speaker/style transfer in speech synthesis.

Resources

The lecture slides, exercise slides, and Unity template are accessible via Moodle.

Tasks

1. Multiple Choice Questions

Please mark the correct answer for the multiple-choice questions below.

a) Text normalization is only needed to expand abbreviations like "Dr." and "Mr."

☐ True ☐ False

b) Machine learning models can be trained to identify whether a period marks the end of a sentence.

☐ True ☐ False

c) Prosodic analysis includes analyzing how loud a person speaks, but not their pitch.

☐ True ☐ False

d) Prominence in speech refers to the volume of words only.

☐ True ☐ False

e) Putting two prominent accents right next to each other is known as a lapse.

☐ True ☐ False

f) One major drawback of diphone synthesis is that it often lacks natural prosody.

☐ True ☐ False

g) Unit selection synthesis can generate more natural-sounding speech than diphone synthesis.

☐ True ☐ False

h) Concatenative synthesis systems create speech by generating sounds from scratch using neural networks.

☐ True ☐ False

i) When comparing a system's generated transcript (SYS) to the correct reference transcript (REF) to calculate WER, a "deletion" error means the system generated an extra word that was not present in the reference.

☐ True ☐ False

j) While the Mean Opinion Score (MOS) requires human listeners to assign an absolute numerical rating to evaluate aspects like expressiveness and naturalness, an A/B Test is a relative evaluation where listeners simply indicate their preference between two different TTS outputs.

☐ True ☐ False

k) To ensure listeners focus solely on distinguishing a single phonetic feature (e.g., “veal” vs. “feel”), the Diagnostic Rhyme Test (DRT) requires that the target words be played in complete isolation without any surrounding context.

☐ True ☐ False

2. Acoustic Models

a) Compare Tacotron 2 and FastSpeech in terms of their architecture and approach to speech synthesis.

1. What are the key differences in how they handle duration modeling, attention, and inference speed?
2. Discuss one advantage and one limitation of each model in practical TTS systems.

b) Tacotron 2 uses Location-Sensitive Attention, while FastSpeech replaces attention entirely by predicting durations explicitly.

1. Explain how attention works in Tacotron 2, including the role of context vectors and learnable parameters.
2. Describe the motivation behind replacing attention with duration prediction in FastSpeech.
3. In what scenarios (e.g., hardware limits, real-time synthesis) might FastSpeech be more appropriate than Tacotron 2?

c) You are given encoder outputs from a FastSpeech model and their corresponding duration predictions: Encoder hidden states (simplified 2D vectors): $h_1 = [0.1, 0.2]$, $h_2 = [0.3, 0.1]$, $h_3 = [0.2, 0.4]$, Duration predictions: $d = [2, 1, 3]$.

1. Perform the Length Regulator operation by repeating each hidden state based on its duration. Write out the resulting sequence as an ordered list of 2D vectors.
2. Briefly explain how this step helps improve the efficiency and reliability of FastSpeech compared to Tacotron 2's autoregressive decoding.

d) Consider the sentence: "He thought it was time to present the present." This sentence contains pronunciation ambiguity due to identical word forms.

1. Describe two challenges this sentence poses for a TTS system.
2. Explain how an acoustic model (e.g., Uni-TTS) could leverage style embeddings, duration prediction, or speaker context to resolve ambiguity in pronunciation and prosody.
3. Suggest one feature (e.g., fine-grained prosody, speaker ID) that would help produce a more natural-sounding output.

3. Vocoders

a) Classical vs Neural Vocoders: Compare Griffin-Lim and WaveNet vocoders in terms of methodology and performance.

1. What input features do each of them require?
2. Discuss one strength and one limitation of each.
3. Why might Griffin-Lim still be used in some scenarios today despite its lower audio quality?

b) HiFiGAN Architecture Analysis: HiFiGAN is a GAN-based neural vocoder with multiple discriminators and a convolutional generator.

1. Briefly explain the roles of the generator, multi-scale discriminator, and multi-period discriminator.
2. Describe how the loss functions (real/fake, feature loss, L1) encourage the generation of realistic and natural-sounding speech.
3. What advantages does HiFiGAN offer compared to autoregressive models like WaveNet?

c) Upsampling and Receptive Fields: In HiFiGAN, upsampling layers are used to convert spectrograms into full-resolution waveforms.

1. Describe what transposed convolutions do in the context of HiFiGAN.
2. The generator uses parallel residual blocks with different dilation rates. Why is this beneficial for speech synthesis?

d) Diffusion-based Vocoder: DiffWave uses a diffusion model instead of autoregression or GANs.

1. Explain the forward and reverse processes in the DiffWave model. What is being learned during training?
2. Discuss the trade-off between generation quality and inference time in DiffWave. Why might someone choose HiFiGAN over DiffWave for real-time TTS?
3. Suggest one practical application where a diffusion vocoder could outperform GANs or classical vocoders.

4. Style Transfer

a) One-Hot Labels vs Learned Embeddings: Text-to-Speech systems can represent speaker/style information using either one-hot labels or learned embeddings.

1. Compare the use of one-hot speaker/style labels and learned embeddings in TTS systems.
2. Discuss one advantage and one limitation of each method with respect to scalability and inference.

b) Speaker Embeddings in Zero-Shot Synthesis: Suppose you train a speaker embedding model using a large speaker-labeled dataset.

1. Explain how a frozen speaker embedding can be used at both training and inference time.
2. Under what condition is zero-shot speaker synthesis possible using this method?
3. What are some benefits of using pretrained foundation models for speaker embeddings?

c) Learned Style Embeddings: Some systems avoid labeling style explicitly and instead learn it directly from audio data.

1. Describe how a reference mel spectrogram is used to infer a style embedding during inference.
2. What are the benefits of using unsupervised style learning (e.g., VAE or token-based) versus manually labeled style categories?
3. Give one example of a real-world application where fine-grained control over prosody or emotion would be especially valuable.

d) System Design Scenario: You are designing a TTS system that can adapt to new speakers with very limited data and provide emotion control (e.g., “happy,” “neutral,” “sad”).

1. Would you choose one-hot or learned embeddings for speaker and style? Justify your answer.
2. What model components or training strategies would you include to support both goals?

Project

Project Stages:

1. **Speech Recognition Integration:** Implement speech-to-text capabilities using OpenAI's Whisper to transcribe spoken words into text. Display the transcriptions either in the console or on the screen.
2. **Large Language Model (LLM) Integration:** Connect a large language model, such as Llama or DeepSeek, to enable the character to generate text-based responses.
3. **Speech Synthesis Integration:** Incorporate speech synthesis technologies like FastSpeech 2 to provide a voice for the character.
4. **Character Animation Enhancement:** Enhance the character with animations, including facial expressions using tools like Audio2Face and body animations through platforms like Mixamo.
5. **Designing and Implementing Dialog Tree:** Create a dialog tree in Twine that links user inputs to narrative paths. Implement it in Rasa by defining intents, slots, and stories to ensure smooth transitions.

Stage 3: Integrating Speech Synthesis

In this stage, you will enable Vincent to audibly respond to the user by integrating a Text-to-Speech (TTS) system using a Python-based backend (e.g., FastSpeech2) and Unity. This involves setting up a Flask server to handle TTS requests and enabling Unity to send text input and play back the generated speech.

1. Choose a TTS System

- Select a TTS model that can generate high-quality speech from text.
- Recommended option: FastSpeech2 + HiFiGAN (locally hosted). They can be downloaded at [FastSpeech2](#)
- Ensure the TTS model is functional and produces .wav files when given text input.

2. Set Up a Flask Server for TTS

- Create or extend a Flask server to include a new endpoint: `/tts` that accepts a POST request with a text field.
- Upon receiving a request:
 - Call your existing FastSpeech2 inference script (e.g., `synthesize.py`).

- Generate a `.wav` audio file from the input text. (Hint: Ensure the generated audio matches Unity’s expected standard parameters to avoid playback issues. E.g., standardizing to 16-bit PCM at 22050 Hz or 44100 Hz.)
- Return the audio file as a response to Unity.

- **Connect Unity to the Flask TTS Endpoint**

- In Unity, use `UnityWebRequest` to:
 - * Send the user’s text input to the Flask server.
 - * Receive the audio file as a byte array.
 - * Save the audio and play it using an `AudioSource` component.
- Ensure proper request formatting using `WWWForm`, and correct handling of file writing and loading via `DownloadHandlerAudioClip`.

- **Automatically Play LLM Responses via TTS**

- After receiving a response from the LLM in Unity following Stage 2, immediately send the response text to the Flask TTS endpoint (`/tts`).
- Use `UnityWebRequest` to:
 - * Send the text to the Flask server.
 - * Receive the audio file as a byte array.
 - * Save the audio and play it using an `AudioSource` component.

(Optional Debugging UI) To test TTS independently from the LLM pipeline, you may also create a simple UI in Unity with:

- `InputField` – to manually enter text.
- `Button` – to trigger speech synthesis manually.

This allows for isolated testing of the TTS server and Unity playback pipeline.

- **Test the TTS Integration**

- Trigger a full interaction: input the user’s speech, receive the LLM response, and confirm it is passed to the TTS system and played in Unity.
- Confirm that:
 - * Flask successfully returns the synthesized speech.
 - * Unity properly plays the returned audio.
 - * Errors (e.g., network, empty input) are handled gracefully.

After completing this stage, Vincent will be able to speak responses aloud, enabling voice-based interaction within Unity.

If you encounter issues, refer to the FastSpeech2 documentation or seek support through the course forum.